

Pflichtenheft: M320 Projekt „Kornav“

Dieses Pflichtenheft wurde mit besserer Formatierung und Korrekturen von Gemini überarbeitet

1. Projektübersicht

- **Projektname:** Kornav
 - **Programmiersprache:** Java (Version 21)
 - **IDE:** IntelliJ IDEA
 - **Dependency-Management:** Maven
 - **Zielplattform:** Linux-Distributionen mit `journalctl` (Hauptfokus / Testumgebung: CachyOS)
 - **Zweck:** Auswerten von System-Logs direkt im Terminal. Kritische Event-Logs werden übersichtlich dargestellt und mögliche Zusammenhänge (Korrelationen) angedeutet.
 - **Programmierungsziel:** OOP Verwenden, Java Convention so gut wie möglich einhalten, Effizienten und Performance-first Code schreiben (Trotzdem Java Native bleiben.)
-

2. Architektonisches Konzept & Technologie-Stack

- **Datenquelle:** Live-Stream von `journalctl -f -o json -n 10` über den `ProcessBuilder`.
 - **Parsing:** Nutzung der **Jackson-Library** zur Konvertierung der JSON-Lines in Event-Klassen/-Objekte (Vermeidung von hoher CPU-Last durch komplexe Regex-Muster).
 - **Multi-Threading & Concurrency:** Einsatz von `LinkedBlockingQueue` und **Caffeine Cache** zur schnellen, thread-sicheren Kommunikation und Datenverarbeitung zwischen den Threads.
 - **Pipeline-Verarbeitung:**
ProcessBuilder (`journalctl -f -o json -n 10`) -> `BufferedReader` -> Jackson (Parsing) -> Logic-Threads -> Correlation-Threads -> Terminal Output
BufferedReader (Vorgefertigte Datei) -> Jackson (Parsing) -> Logic-Threads -> Correlation-Threads -> Terminal Output
-

3. Geplante Features

- **Picocli:** Parameter übergabe für zwecke wie externe file output zum logging

- **WICHTIG:** mit dem Parameter -test "pfad/zur/txt" ändert sich die Pipeline anstelle Processbuilder , BufferedReader mit sleep zu gebrauchen was journalctl "simuliert" und Ausführung auf Windows oder Mac ermöglicht, die datei muss ein txt sein aber muss die Daten per line im json format haben.
 - **Simples"TUI" Display** mit gezielten ausgaben:
 - Error per Log rate (%)
 - Logs pro 10 Sekunden
 - Erschlüsse der Korrelationsengine
 - **Robusten Code:** so gut wie möglich alle Exceptions auffinden und selbst "reparierenden" Code schreiben so das wenn möglicherweise als Beispiel Processbuilder stoppt das Programm dies von allein wieder auf starten kann.
-

4. Optionale Features

- **Farben:** TUI wird mit Farben ergänzt
- **Software Performance Monitor:** zeigt im TUI noch Information von RAM Usage Queue Auslastung und Effizienz der generellen Pipeline
- **Incident Logger:** speichert in einem unterordner dumps ab die vor einem Kritischem Event passiert sind ab
- **GraalVM:** vorgefertige Binary/Executable Datei im Repo mitgeliefert (Windows & Linux)